```
  end

  def _singleton_class
    class << self
      self
    end
  end

  def method_missing(name, *args, &block)
    if name[-1] == "="
      base_name = name[0..-2].intern
      _singleton_class.instance_exec(name) do |name|
        define_method(name) do |value|
          @values[base_name] = value
        end
      end
      @values[base_name] = args[0]
    else
      _singleton_class.instance_exec(name) do |name|
        define_method(name) do
          @values[name]
        end
      end
      @values[name]
    end
  end

  def respond_to_missing?(_)
    true
  end
end

obj = MyOpenStruct.new(name: "Dave")
obj.address = "Texas"
obj.likes = "Programming"

puts "#{obj.name} lives in #{obj.address} and likes #{obj.likes}"
```

produces:

```
Dave lives in Texas and likes Programming
```

Notice how we base our class on BasicObject. BasicObject is the root of Ruby's object hierarchy and contains only a minimal number of methods:

```
p BasicObject.instance_methods
```

produces:

```
[:!, :equal?, :__send__, :==, :!=, :instance_eval, :instance_exec, :__id__]
```

This is good, because it means that our MyOpenStruct class will be able to have attributes such as display or class. If instead we'd based MyOpenStruct on class Object, then these names, along with forty-nine others, would have been predefined and hence wouldn't trigger method_missing.

Notice also another common pattern inside method_missing. The first time we reference or assign to an attribute of our object, we access or update the @values hash appropriately. But we also use define_method to define the method that the caller was trying to access. This means that the next time this attribute is used, it will use the method and not invoke method_missing.